SECURE DATA SHARING USING RSA ENCRYPTION

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MASTER OF COMPUTER APPLICATIONS

BY

AVINASH BALAKRISHNAN

REG. NO. DC2332306010017

UNDER THE GUIDANCE OF

Dr. A. THIRUMURTHI RAJA M.C.A., M.E., PH.D.,

ASSISTANT PROFESSOR,



DEPARTMENT OF COMPUTER APPLICATIONS

DIRECTORATE OF DISTANCE EDUCATION

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be university u/s 3 of UGC Act, 1956)

Kattankulathur – 603 203

Chennai, Tamilnadu

JUNE 2024

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"SECURE DATA SHARING USING RSA ENCRYPTION"** is a bonafide work carried out by **AVINASH BALAKRISHNAN** (**DC2232306010017)** under my supervision for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by these students.

PROJECT GUIDE

Dr.A.Thirumurthi Raja

Assistant Professor,

SRMIST-DDE,

HEAD OF DEPARTMENT

Dr.A.Thirumurthi Raja

Program Coordinator

Department of Computer Applications,

Directorate of Distance Education (DDE)

SRMIST,

INTERNAL EXAMINER

EXTERNAL EXAMINER

`

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words "ANYHING IS ACHIEVABLE WITH HARDWORK AND PERSISTENCE".

I am thankful to **Dr.T.R. Paarivendhar, Chancellor**, SRM Institute of Science & Technology who gave us the platform to establish me to reach greater heights.

I express my deep sense of gratitude to the Director **Dr. R. Rajagopal** for their wholehearted support and encouragement.

With gratitude I like to thank my project guide **Dr.A.Thirumurthi Raja** Assistant Professor, Department of Computer Applications (DDE), for his help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

I take this opportunity to extend this gratitude to all the faculty members of the department who provided their support through valuable comments and suggestions during the reviews.

A great note of gratitude to friends and people who are known to me who helped in carrying out this project work a successful one.

Avinash Balakrishnan

TABLE OF CONTENTS

1. CHAPTER 1 –

1.0 INTRODUCTION……..6

1.1 EXISTING SYSTEM…….8

1.2 PROPOSED SYSTEM……...9

1.3 SYSTEM SPECIFICATION……..10

1.3.1 HARDWARE REQUIREMENTS

1.3.2 SOFTWARE REQUIREMENTS

1.4 ARCHITECTURE DIAGRAM…….11

1.5 UML DIAGRAMS……12

1.5.1 USE CASE DIAGRAM…… 13

1.5.2 CLASS DIAGRAM …….14

1.5.3 SEQUENCE DIAGRAM…….15

1.5.4 ACTIVITY DIAGRAM……...16

1.5.5 DATAFLOW DIAGRAM……17

2. CHAPTER 2 –

2.1 LITERATURE SURVEY……..18

3. CHAPTER 3 –

3.0 MODULES……22

3.1 MODULE DESCRIPTION……..23

3.1.1 DATA OWNER……..23

3.1.2 DATA USER……..24

3.1.3 ADMIN……...25

4. CHAPTER 4 –

4.0 LANGUAGE SPECIFICATIONS…….27

5. CHAPTER 5 –

5.1 SYSTEM TESTING……...54

5.1.1 SYSTEM TESTING AND MAINTENANCE……..56

5.2 UNIT TESTING…….57

5.3 INTEGRATION TESTING……58

6. CHAPTER 6 –

6.1 CONCLUSION……...59

6.2 SCREENSHOTS……60

6.3 CODING…….64

6.4 REFERENCES……81

Secure Data Sharing Platform with RSA Encryption

Abstract:

In the digital age, the proliferation of data necessitates robust mechanisms to safeguard sensitive information while facilitating controlled access. This project addresses this imperative by introducing a secure data sharing platform employing RSA encryption. The platform caters to three primary entities: data owners, data users, and administrators, each with distinct roles and privileges.Data owners are empowered to upload files onto the platform, with each text file being encrypted using the RSA algorithm prior to storage. This encryption ensures that even if unauthorized access to the database occurs, the contents remain indecipherable without the requisite decryption keys.Data users, on the other hand, can browse and access files uploaded by data owners. However, access to files is restricted until permission is granted by the respective data owner. To access a particular file, a data user submits a request to the data owner. Upon approval, the data owner securely transmits the decryption key to the requesting user, enabling them to decrypt and download the file. This granular control over access ensures that sensitive data remains protected, and access is granted only to authorized individuals.The platform also features an administrative component tasked with overseeing user management and ensuring the integrity of the system. Administrators are responsible for user authentication, monitoring user activity, and resolving any disputes or security breaches that may arise.By integrating RSA encryption with controlled access mechanisms, this platform provides a comprehensive solution for secure data sharing. It empowers data owners to maintain control over their information while enabling seamless collaboration and exchange between authorized users. Moreover, the robust encryption schemensures data confidentiality, mitigating the risk of unauthorized access or data breaches.

CHAPTER 1

1.0 INTRODUCTION:

In today's digital landscape, the exponential growth of data brings forth both opportunities and challenges. While the accessibility and abundance of information foster innovation and collaboration, they also raise concerns regarding data security and privacy. The imperative to safeguard sensitive information while facilitating controlled access has become paramount.Addressing this imperative, a pioneering project emerges, introducing a secure data sharing platform fortified by RSA encryption. This innovative platform caters to the evolving needs of modern data management, offering robust mechanisms to protect sensitive data while enabling seamless collaboration among stakeholders.Within this framework, three primary entities play integral roles: data owners, data users, and administrators. Each entity is equipped with distinct roles and privileges, fostering a structured and secure environment for data sharing.Data owners, entrusted with the stewardship of sensitive information, wield the power to upload files onto the platform securely. Leveraging the RSA algorithm, each text file undergoes encryption before storage, ensuring that even in the event of unauthorized access, the contents remain indecipherable without the requisite decryption keys.Conversely, data users are granted controlled access to files uploaded by data owners. However, access is not granted indiscriminately; instead, it is contingent upon approval from the respective data owner. Through a streamlined process, data users submit requests for access, which are diligently reviewed by data owners. Upon approval, the decryption key is securely transmitted to the requesting user, facilitating the decryption and download of the file. This granular control over access ensures that sensitive data remains safeguarded, accessible only to authorized individuals. Complementing these user-centric functionalities is the administrative component, tasked with upholding the

integrity of the system. Administrators play a pivotal role in user authentication, monitoring user activity, and swiftly addressing any disputes or security breaches that may arise, thereby fortifying the platform's resilience against potential threats.By seamlessly integrating RSA encryption with controlled access mechanisms, this platform emerges as a beacon of innovation, offering a comprehensive solution for secure data sharing in the digital age. It empowers data owners to maintain sovereignty over their information while facilitating seamless collaboration and exchange among authorized users. Moreover, the robust encryption scheme underpinning the platform ensures data confidentiality, mitigating the risk of unauthorized access or data breaches.In essence, this project represents a paradigm shift in data management, ushering in a new era of secure and collaborative data sharing underpinned by cutting-edge encryption technologies.

1.1 EXISTING SYSTEM:

An existing system that closely aligns with the described project is a secure file sharing platform with encryption capabilities. One example of such a system is Dropbox for Business, particularly when it's augmented with additional encryption layers. Dropbox for Business provides a platform for users to upload and share files securely. It offers features such as access controls, permissions management, and user authentication to ensure that only authorized individuals can access sensitive data.

DISADVANTAGES:

An existing system like Dropbox for Business, even with additional encryption layers. While platforms like Dropbox invest heavily in security measures, they still represent a third-party entity responsible for safeguarding sensitive data. Depending solely on such platforms introduces a level of dependency and trust on the service provider's security practices and infrastructure. Any vulnerabilities or breaches in the service provider's systems could compromise the security of the encrypted files, regardless of the additional encryption layers applied by the users. In many third-party file sharing platforms, users may not have full control over encryption keys. This lack of control means that users must trust the platform to manage and safeguard the keys effectively. In scenarios where users require absolute control over encryption keys for compliance or regulatory reasons, relying on a third-party platform may not be suitable.

1.2 PROPOSED SYSTEM:

The proposed system is a custom-built secure data sharing platform designed to meet the specific requirements outlined in the project description. The system integrates a custom encryption module based on the RSA algorithm to encrypt files uploaded by data owners before storing them on the platform. This ensures that all sensitive data remains secure, even if unauthorized access to the platform occurs. The platform features robust user authentication mechanisms to verify the identities of data owners, data users, and administrators. Access control mechanisms are implemented to ensure that only authorized users can upload, access, and manage files.

ADVANTAGES :

- The proposed system offers enhanced security compared to existing third-party platforms by incorporating custom encryption mechanisms tailored to the organization's specific security requirements. By encrypting files before they are uploaded to the platform and controlling access through a request and approval workflow, the system mitigates the risk of unauthorized access and data breaches.
- The platform provides data owners with granular control over who can access their files. Data owners can selectively grant access to authorized users based on their specific needs and permissions, ensuring that sensitive information remains protected while enabling collaboration and data sharing among trusted parties.

1.3 SYSTEM SPECIFICATION

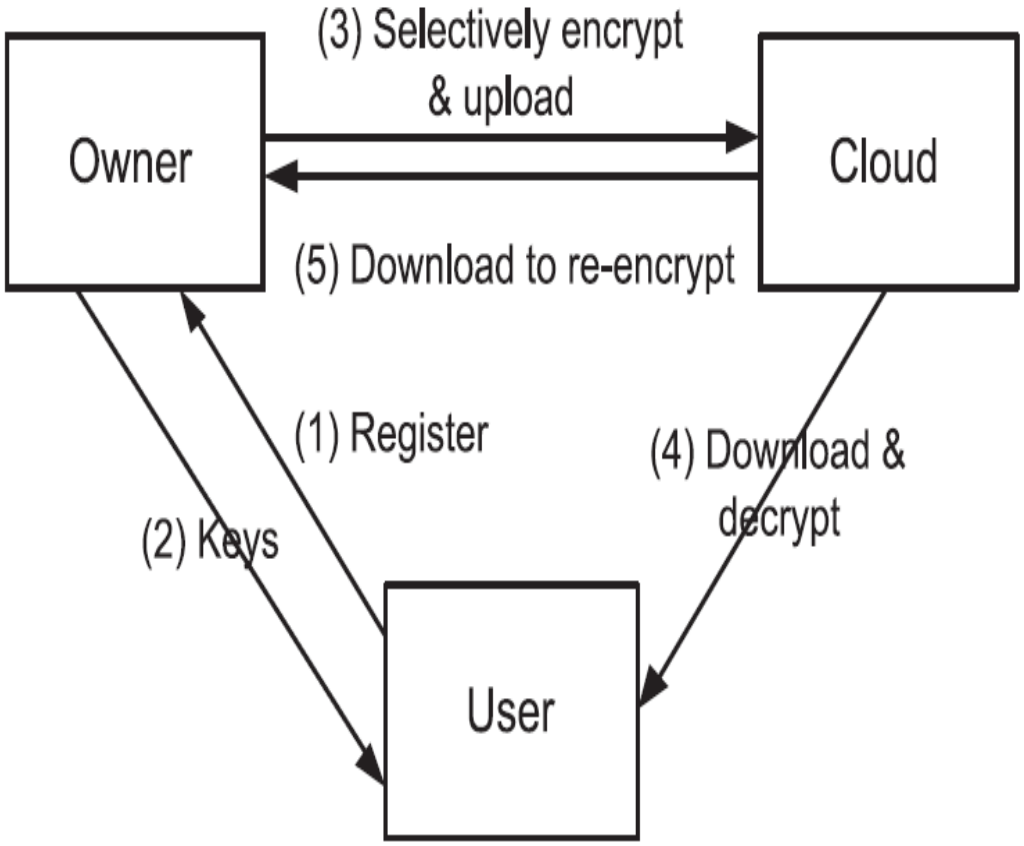
1.3.1 HARDWARE REQUIREMENTS:

- Processor : Intel Core i3 Processor
- Speed : 2.5 GHz
- RAM : 2GB(min)
- Hard Disk : 500MB
- Key Board : Standard Windows Keyboard
- Mouse : Two or Three Button Mouse
- Monitor : LCD

1.3.2 SOFTWARE REQUIREMENTS:

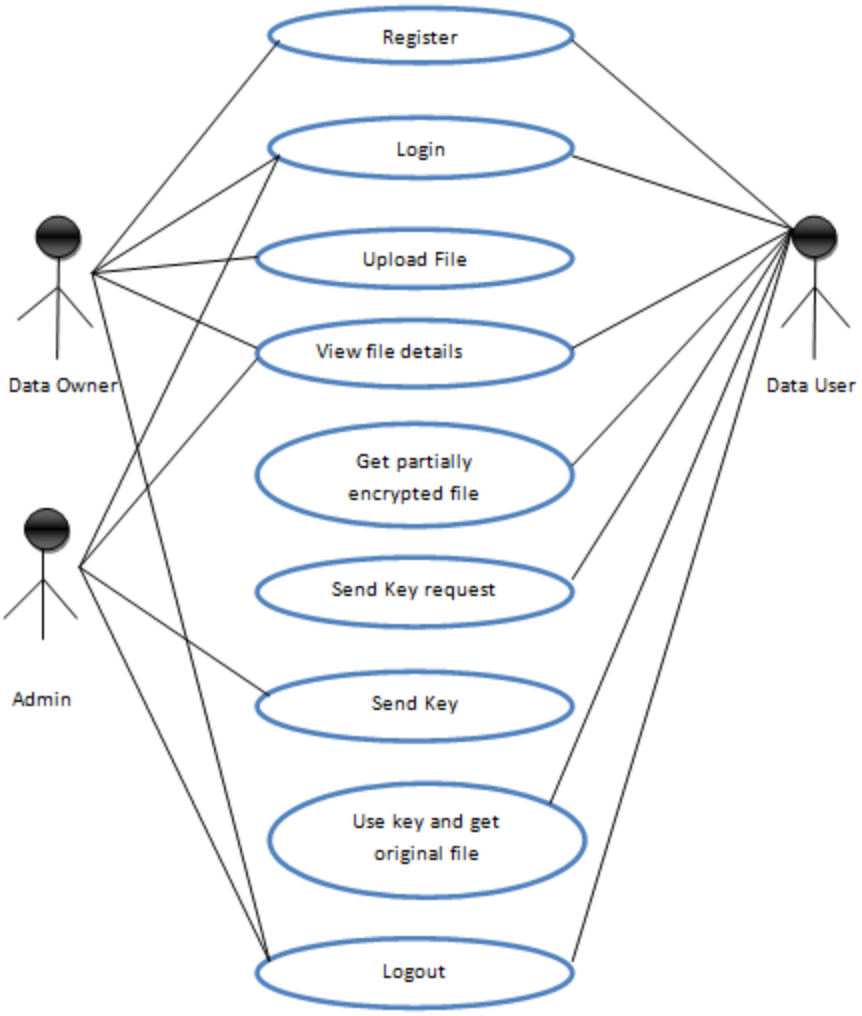
- Operating System : Windows7/10.
- Application Server : Tomcat6.0/7/8.X.
- Front End : Java , HTML,CSS
- Scripts : JavaScript.
- Server side Script : Java Server Pages.
- IDE : Net beans
- Back End : MYSQL 5.0/ Heidi SQL 8.1
- Database Connectivity : JDBC

1.4 ARCHITECTURE DIAGRAM:

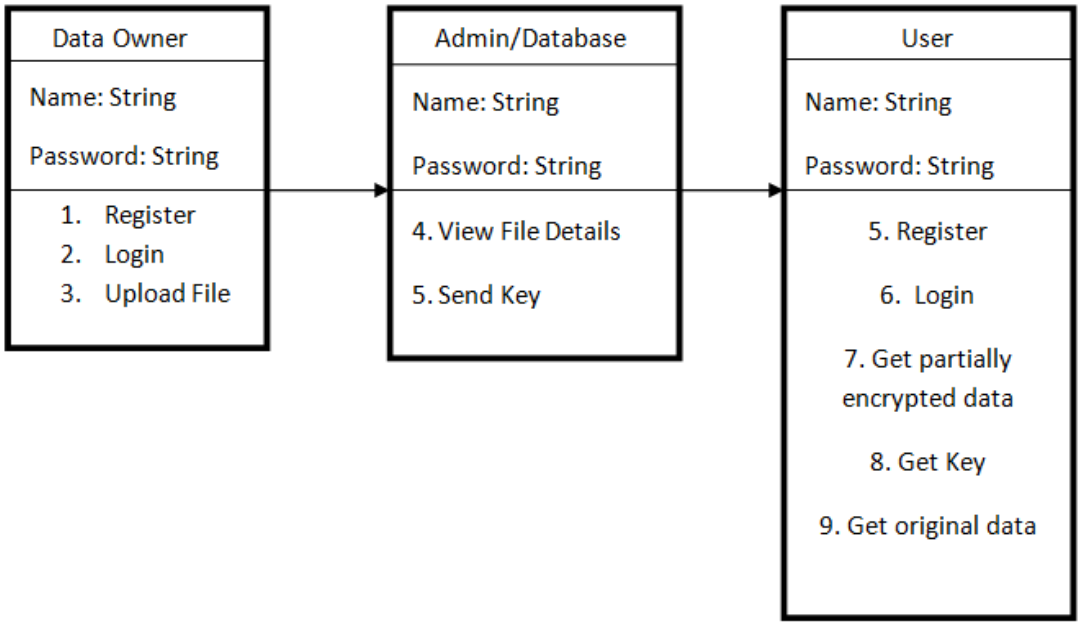


1.5 UML DIAGRAMS:

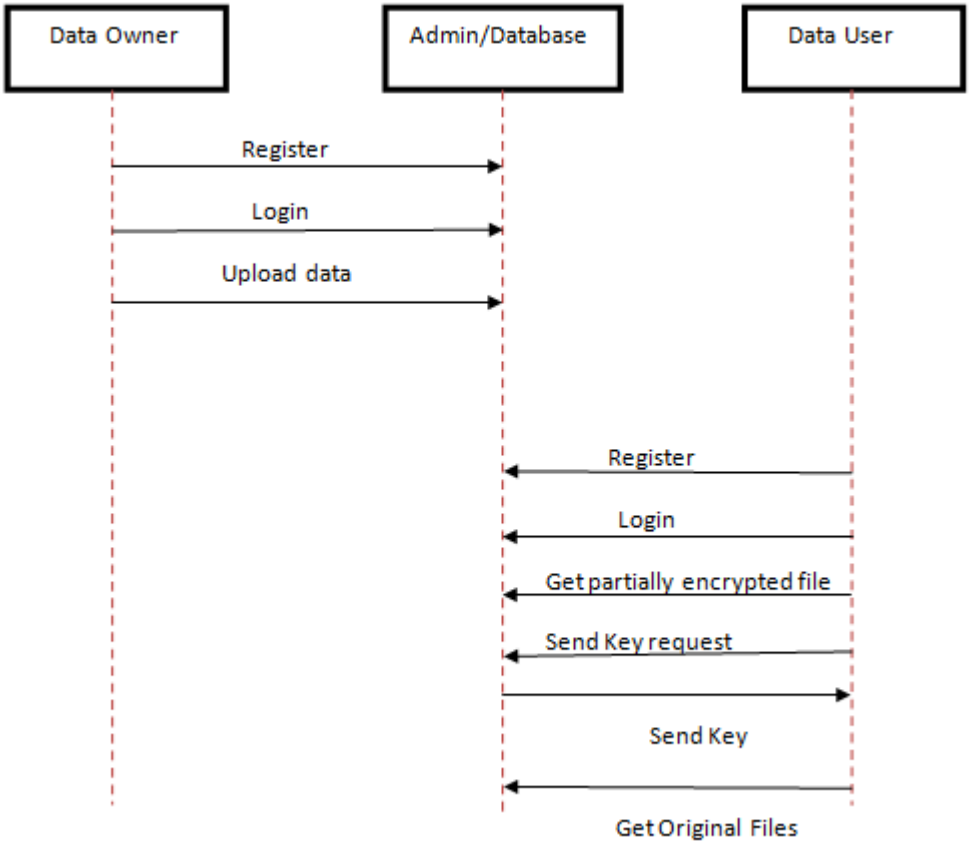
1.5.1 Use Case Diagram:



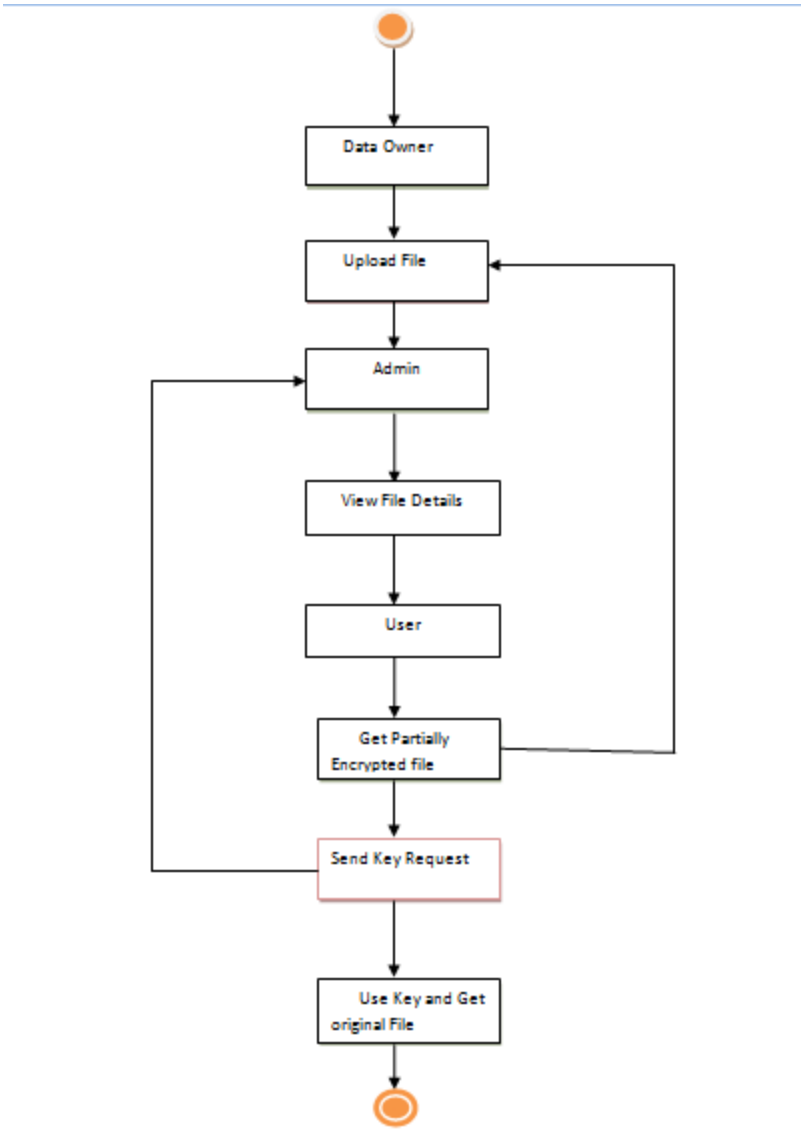
1.5.2 Class Diagram:



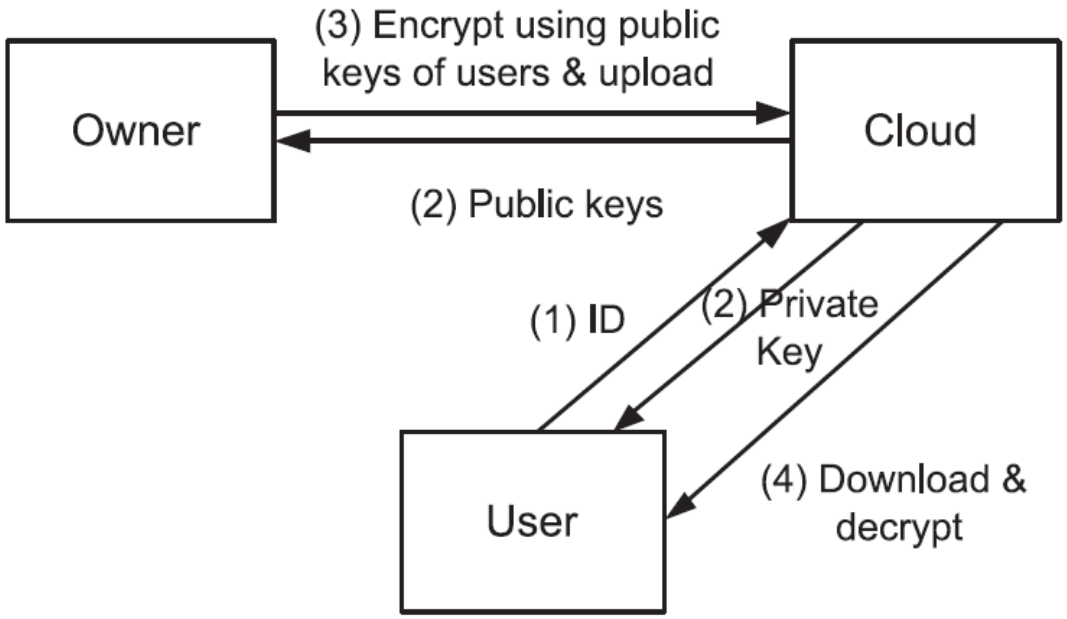
1.5.3 Sequence Diagram:



1.5.4 Activity Diagram:



1.5.5 DATAFLOW DIAGRAM



CHAPTER 2

2.1 LITERATURE SURVEY:

TITLE: Certificateless public key cryptography

YEAR OF PUBLISHING: 2020

AUTHOR NAME: K.Paterson

ABSTRACT: This paper introduces and makes concrete the concept of certificateless public key cryptography (CL-PKC), a model for the use of public key cryptography which avoids the inherent escrow of identity-based cryptography and yet which does not require certificates to guarantee the authenticity of public keys. The lack of certificates and the presence of an adversary who has access to a master key necessitates the careful development of a new security model. We focus on certificateless public key encryption (CL-PKE), showing that a concrete pairing-based CL-PKE scheme is secure provided that an underlying problem closely related to the Bilinear Diffie-Hellman Problem is hard.

TITLE: Searchable encryption revisited: Consistency properties, relation to anonymousibe, and extensions
YEAR OF PUBLISHING: 2020

AUTHOR NAME: M.Abdalla

ABSTRACT: We identify and fill some gaps with regard to consistency (the extent to which false positives are produced) for public-key encryption with keyword search (PEKS). We define computational and statistical relaxations of the existing notion of perfect consistency, show that the scheme of is computationally consistent, and provide a new scheme that is statistically consistent. We also provide a transform of an anonymous IBE scheme to a secure PEKS scheme that, unlike the previous one, guarantees consistency. Finally we suggest three extensions of the basic notions considered here, namely anonymous HIBE, public-key encryption with temporary keyword search, and identity-based encryption with keyword search.

TITLE: Relations among notions of security for public-key encryption schemes

YEAR OF PUBLISHING: 2020

AUTHOR NAME: A.Desai

ABSTRACT: We compare the relative strengths of popular notions of security for public key encryption schemes. We consider the goals of privacy and non-malleability, each under chosen plaintext attack and two kinds of chosen ciphertext attack. For each of the resulting pairs of definitions we prove either an implication (every scheme meeting one notion must meet the other) or a separation (there is a scheme meeting one notion but not the other, assuming the first notion can be met at all). We similarly treat plaintext awareness, a notion of security in the random oracle model. An additional contribution of this paper is a new definition of non-malleability which we believe is simpler than the previous one.

TITLE: Fine-grained control of security capabilities

YEAR OF PUBLISHING: 2020

AUTHOR NAME: X.Ding

ABSTRACT: Recently, Libert and Quisquater showed that the fast revocation method using a SEcurity Mediator(SEM) in a mRSA can be applied to the Boneh-Franklin identity based encryption and GDH signature scheme. In this paper we propose a mediated identity based signature(mIBS) scheme which applies the SEM architecture to an identity based signature. The use of a SEM offers a number of practical advantages over current revocation techniques. The benefits include simplified validation of digital signatures, efficient and fast revocation of signature capabilities. We further propose a forward mediated signature scheme with an efficient batch verification and analyze their security and efficiency

TITLE: Controlling access to an oblivious database using stateful anonymous credentials

YEAR OF PUBLISHING: 2020

AUTHOR NAME: M.Green

ABSTRACT: In this work, we consider the task of allowing a content provider to enforce complex access control policies on oblivious protocols conducted with anonymous users. As our primary application, we show how to construct privacy-preserving databases by combining oblivious transfer with an augmented anonymous credential system. This permits a database operator to restrict which items each user may access, without learning anything about users' identities or item choices. This strong privacy guarantee holds even when users are assigned different access control policies and are allowed to adaptively make many queries. To do so, we show how to augment existing anonymous credential systems so that, in addition to certifying a user's attributes, they also store state about the user's database access history. Our construction supports a wide range of access control policies, including efficient and private realizations of the Brewer-Nash (Chinese Wall) and Bell-LaPadula (Multilevel Security) policies, which are used for financial and defense applications. In addition, our system is based on standard assumptions in the standard model and, after an initial setup phase, each transaction requires only constant time.

CHAPTER 3

MODULES

1. Data Owner
2. Data User
3. Admin

3.1 MODULE DESCRIPTION:

3.1.1 Data Owner:

The Data Owner module in the proposed secure data sharing platform plays a crucial role in controlling and managing the data shared on the platform. It encompasses functionalities related to uploading, encrypting, granting access, and managing files stored on the platform. Below is a detailed description of the functionalities included in the Data Owner module:

File Upload: Data owners have the capability to upload files onto the platform. This functionality enables data owners to share their data securely with authorized users while maintaining control over its access and integrity.

Encryption: Before uploading files to the platform, data owners can choose to encrypt them using strong cryptographic algorithms such as AES. This ensures that the contents of the files remain confidential and indecipherable to unauthorized parties, even if the platform's security is compromised.

Access Control: Data owners have the authority to control access to the files they upload. They can specify which users or user groups are permitted to access each file, thereby ensuring that sensitive information is shared only with authorized individuals or entities.

Request Management: When a data user requests access to a file uploaded by a data owner, the Data Owner module facilitates the management of these access requests. Data owners can view, approve, or deny access requests based on their discretion and the nature of the requested file.

Decryption Key Management: In scenarios where access requests are approved, data owners are responsible for securely providing decryption keys to authorized users. This ensures that only authorized users can decrypt and access the encrypted files, maintaining the confidentiality and integrity of the data.

File Revocation: Data owners have the ability to revoke access to files at any time. This functionality is useful in situations where access privileges need to be revoked due to changes in permissions, data ownership, or security concerns.

3.1.2 Data User:

The Data User module in the proposed secure data sharing platform facilitates access to encrypted files uploaded by data owners. Data users interact with the platform to request access to specific files, receive decryption keys, and securely download decrypted files for authorized use.

Access Request Submission: When a data user identifies a file of interest, they can submit an access request to the data owner who uploaded the file. Access requests typically include a justification for why the user requires access to the file and any additional information requested by the data owner.

Access Request Tracking: The Data User module includes features for tracking the status of access requests submitted by the data user. Data users can monitor the

progress of their requests, including whether they are pending, approved, or denied by the data owner.

Decryption Key Reception: Upon approval of an access request by the data owner, the Data User module facilitates the reception of decryption keys from the data owner. Decryption keys are securely transmitted to the data user, enabling them to decrypt and access the requested file.

File Download: With the received decryption key, data users can securely download and access the decrypted file from the platform. This functionality ensures that only authorized users with the appropriate decryption keys can access the sensitive information contained within the file.

3.1.3 Admin:

The Administrator module in the proposed secure data sharing platform serves as the central authority responsible for overseeing and managing the platform's operations, users, and files. Administrators have privileged access to view, monitor, and administer data owner details, data user details, and file details.

User Management: Administrators have the authority to manage user accounts, including data owners and data users, within the platform. This functionality enables administrators to create, update, delete, or suspend user accounts as necessary. Additionally, administrators can assign roles, permissions, and access levels to users based on their responsibilities and requirements.

Data Owner Details: Administrators can view and manage details related to data owners registered on the platform. This includes information such as user profiles, contact details, organizational affiliations, and permissions. Administrators may also

have the ability to audit and verify data owner credentials or certifications, ensuring the integrity and legitimacy of user accounts.

Data User Details: Similarly, administrators can access and oversee details associated with data users registered on the platform. This includes user profiles, contact information, access permissions, and activity logs. Administrators may also monitor user behavior and compliance with platform policies to ensure responsible data usage and adherence to security protocols.

File Details: Administrators have visibility into the files uploaded and shared on the platform by data owners. This includes metadata such as file names, sizes, upload dates, encryption status, and access permissions. Administrators may review file access logs, audit trails, and access control policies to identify any anomalies or security incidents**.**

CHAPTER 4

4.0 LANGUAGE SPECIFICATIONS:

Java Technology

Java technology is both a programming language and a platform.

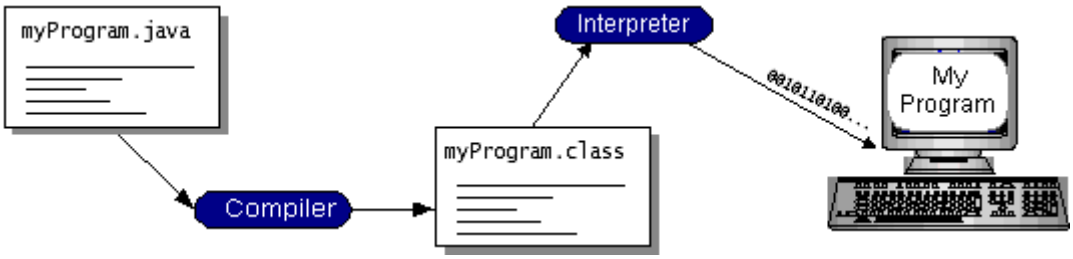
The Java Programming Language

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

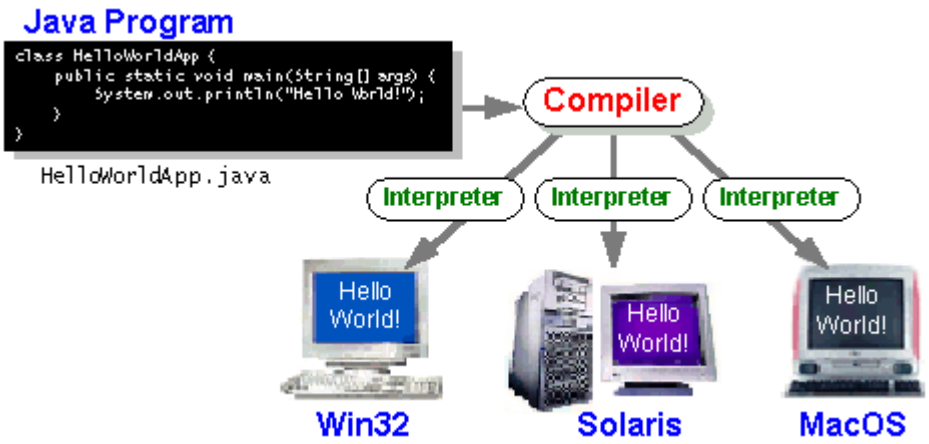
- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called *Java byte codes* —the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed.

The following figure illustrates how this works.



You can think of Java bytecodes as the machine code instructions for the *Java Virtual Machine* (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java bytecodes help make "write once, run anywhere" possible. You can compile your program into bytecodes on any platform that has a Java compiler. The bytecodes can then be run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.



The Java Platform

A *platform* is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

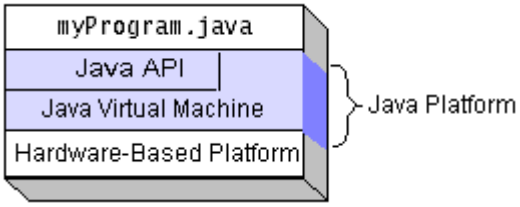
- *The Java Virtual Machine (Java VM)*
- *The Java Application Programming Interface* (Java API)

You've already been introduced to the Java VM. It's the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as *packages*. The next section, What Can

Java Technology Do?.Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.



Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time bytecode compilers can bring performance close to that of native code without threatening portability.

What Can Java Technology Do?

The most common types of programs written in the Java programming language are *applets* and *applications*. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

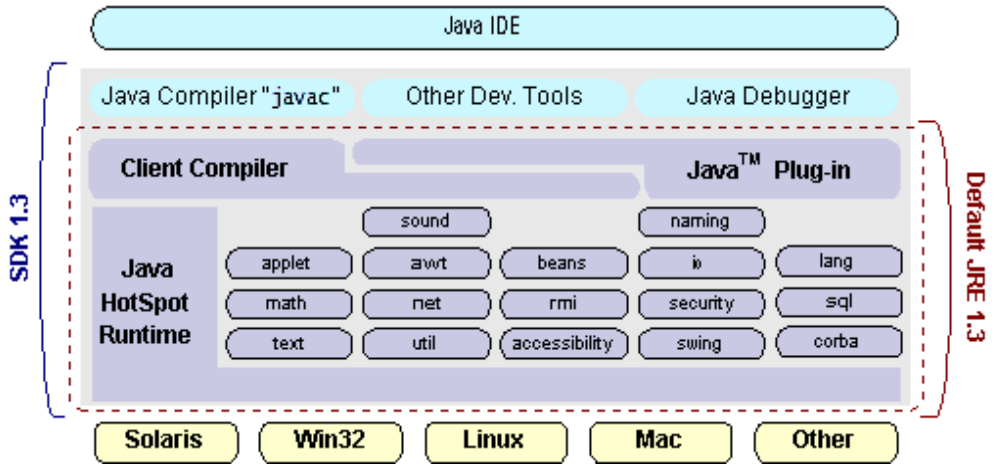
An application is a standalone program that runs directly on the Java platform. A special kind of application known as a *server* serves and supports clients on

a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a *servlet*. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials**: Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets**: The set of conventions used by applets.
- **Networking**: URLs, TCP (Transmission Control Protocol), UDP (User Data gram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization**: Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.
- **Security**: Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components**: Known as JavaBeans™, can plug into existing component architectures.
- **Object serialization**: Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBC™)**: Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.



How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly**: Although the Java programming language is a powerful object-oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- **Write less code**: Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- **Write better code**: The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory

leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.

- **Develop programs more quickly**: Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.
- **Avoid platform dependencies with 100% Pure Java**: You can keep your program portable by avoiding the use of libraries written in other languages. The 100% Pure Java ™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.
- **Write once, run anywhere**: Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java platform.
- **Distribute software more easily**: You can upgrade applets easily from a central server. Applets take advantage of the feature

ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a *de facto* standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a MYSQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as MYSQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data Sources.

From a programming perspective, the beauty of ODBC is that the application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or MYSQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or MYSQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server

environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

JDBC

In an effort to set an independent database standard API for Java, Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic MYSQL database access mechanism that provides a consistent interface to a variety of RDBMS. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or *drivers*. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution.

JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

JDBC Goals

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java.

The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do.

The design goals for JDBC are as follows:

1. **MYSQL Level API**

   The designers felt that their main goal was to define a MYSQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently.

2. **MYSQL Conformance**

   MYSQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This

allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

3. **JDBC must be implemental on top of common database interfaces**

The JDBC MYSQL API must "sit" on top of other common MYSQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

4. **Provide a Java interface that is consistent with the rest of the Java system**

Because of Java's acceptance in the user community thus far, the designers feel that they should not stray from the current design of the core Java system.

5. **Keep it simple**

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

6. **Use strong, static typing wherever possible**

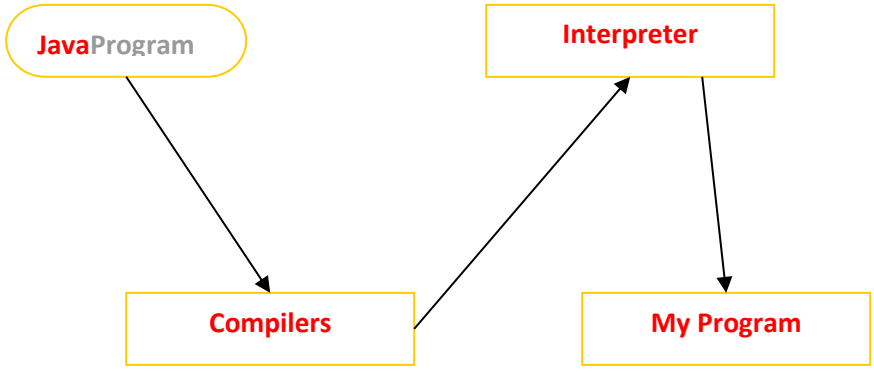
Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

7. **Keep the common cases simple**

Because more often than not, the usual MYSQL calls used by the programmer are simple SELECT's, INSERT's, DELETE's and UPDATE's, these queries should be simple to perform with JDBC. However, more complex MYSQL statements should also be possible.

Finally we decided to precede the implementation using JavaNetworking. And for dynamically updating the cache table we go for MS Access database.

Java is also unusual in that each java program is both compiled and interpreted. With a compile you translate a java program into an intermediate language called java byte codes the platform-independent code instruction is passed and run on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The figure illustrates how this works.



You can think of java byte codes as the machine code instructions for the java virtual machine (java vm). Every java interpreter, whether it's a java development tool or a web browser that can run java applets, is an implementation of the java vm. The java vm can also be implemented in hardware. Java byte codes help make "write once, run anywhere" possible. You can compile your java program into byte codes on my platform that has a java compiler. The byte codes can then be run any implementation of the java vm. For example, the same java program can run windows NT, solaris, and macintosh.

Netbeans:

NetBeans is an integrated development environment (IDE) for developing primarily with Java, but also with other languages, in particular PHP, C/C++, and HTML5. It is also an application platform framework for Java desktop applications and others. The NetBeans IDE is written in Java and can run on Windows, OS X, Linux, Solaris and other platforms supporting a compatible JVM.

The NetBeans Platform allows applications to be developed from a set of modular software components called modules. Applications based on the NetBeans Platform (including the NetBeans IDE itself) can be extended by third party developers.[1]

NetBeans Platform Features:

The main reusable features and components comprising the NetBeans Platform are outlined below.

Module System:

The modular nature of a NetBeans Platform application gives you the power to meet complex requirements by combining several small, simple, and easily tested modules encapsulating coarsely-grained application features. Powerful versioning support helps give you confidence that your modules will work together, while strict control over the public APIs your modules expose will help you create a more flexible application that's easier to maintain. Since your application can use either standard NetBeans Platform modules or OSGi bundles, you'll be able to integrate third-party modules or develop your own.

Lifecycle Management:

Just as application servers, such as GlassFish or Web Logic, provide lifecycle services to web applications, the NetBeans runtime container provide lifecycle services to Java desktop applications. Application servers understand how to compose web modules, EJB modules, and related artifacts, into a single web application.

In a comparable manner, the NetBeans runtime container understands how to compose NetBeans modules into a single Java desktop application.

There is no need to write a main method for your application because the NetBeans Platform already contains one. Also, support is provided for persisting user settings across restart of the application, such as, by default, the size and positions of the windows in the application.

Plug ability, Service Infrastructure, and File System:

End users of the application benefit from pluggable applications because these enable them to install modules into their running applications. NetBeans modules can be installed, uninstalled, activated, and deactivated at runtime, thanks to the runtime container. The NetBeans Platform provides an infrastructure for registering and retrieving service implementations, enabling you to minimize direct dependencies between individual modules and enabling a loosely coupled architecture (high cohesion and low coupling).

The NetBeans Platform provides a virtual file system, which is a hierarchical registry for storing user settings, comparable to the Windows Registry on Microsoft Windows systems. It also includes a unified API providing stream-oriented access

to flat and hierarchical structures, such as disk-based files on local or remote servers, memory-based files, and even XML documents.

Window System, Standardized UI Toolkit, and Advanced Data-Oriented Components:

Most serious applications need more than one window. Coding good interaction between multiple windows is not a trivial task. The NetBeans window system lets you maximize/minimize, dock/undock, and drag-and-drop windows, without you providing any code at all. Swing and JavaFX are the standard UI toolkits on the Java desktop and can be used throughout the NetBeans Platform. Related benefits include the ability to change the look and feel easily via "Look and Feel" support in Swing and CSS integration in JavaFX, as well as the portability of GUI components across all operating systems and the easy incorporation of many free and commercial third-party Swing and JavaFX components.

With the NetBeans Platform you're not constrained by one of the typical pain points in Swing: the JTree model is completely different to the JList model, even though they present the same data. The NetBeans Nodes API provides a generic model for presenting your data. The NetBeans Explorer & Property Sheet API provides several advanced Swing components for displaying nodes. In addition to a window system, the NetBeans Platform provides many other UI-related components, such as a property sheet, a palette, and complex Swing components for presenting data, a Plug-in Manager, and an Output window.

Miscellaneous Features, Documentation, and Tooling Support:

The NetBeans IDE, which is the software development kit (SDK) of the NetBeans Platform, provides many templates and tools, such as the award winning Matisse GUI Builder that enables you to very easily design your application's layout.

The NetBeans Platform exposes a rich set of APIs, which are tried, tested, and continually being improved. The community is helpful and diverse, while a vast library of blogs, books, tutorials, and training materials are continually being developed and updated in multiple languages by many different people around the world.

Apache Tomcat:

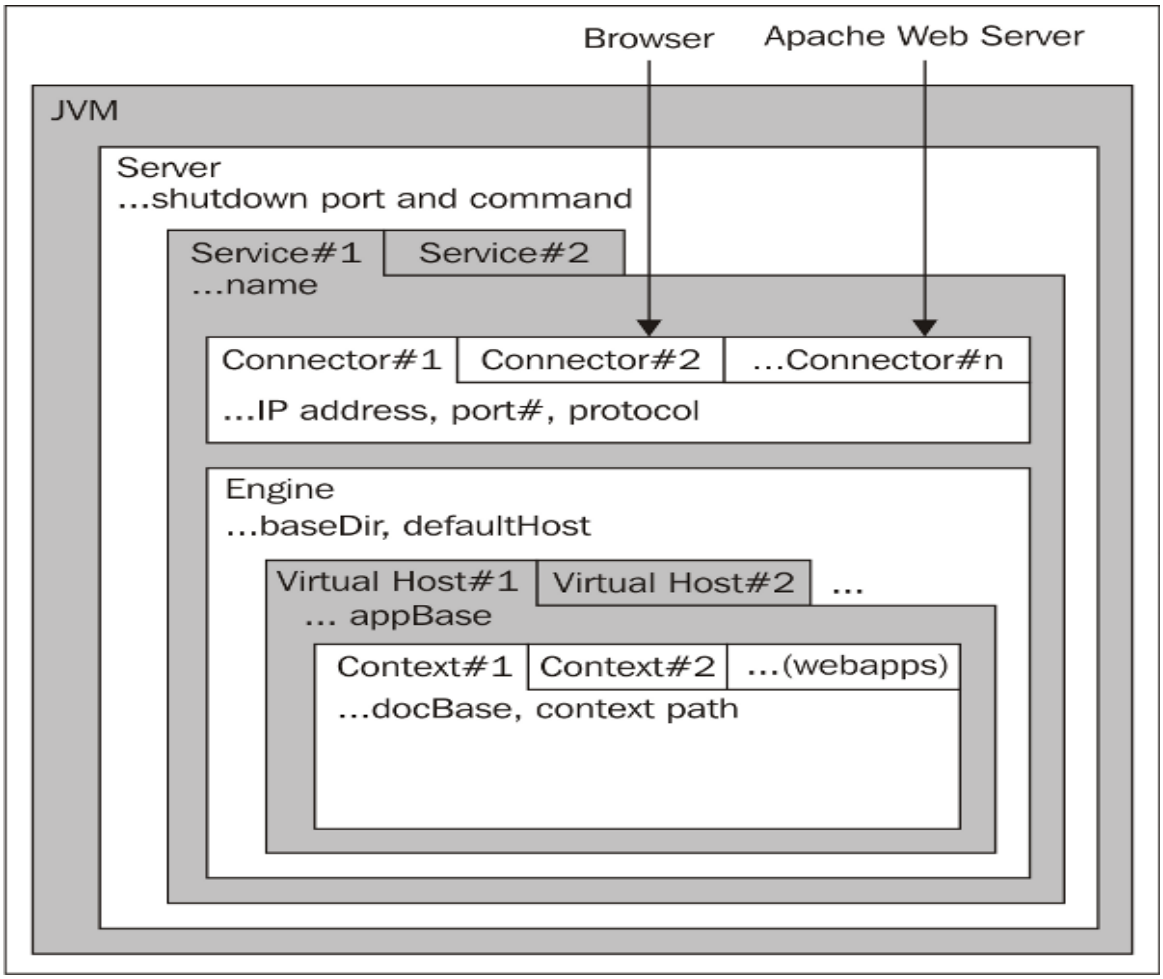
Apache Tomcat (or simply Tomcat, formerly also Jakarta Tomcat) is an open source web server and servlet container developed by the Apache Software Foundation (ASF). Tomcat implements the Java Servlet and the Java Server Pages (JSP) specifications from Oracle Corporation, and provides a "pure Java" HTTP web server environment for Java code to run.

Component taxonomy:

Tomcat's architecture follows the construction of a Matrushka doll from Russia. In other words, it is all about containment where one entity contains another, and that entity in turn contains yet another. In Tomcat, a 'container' is a generic term that refers to any component that can contain another, such as a Server, Service, Engine, Host, or Context. Of these, the Server and Service components are special containers, designated as Top Level Elements as they represent aspects of the running Tomcat instance. All the other Tomcat components are subordinate to these top level elements. The Engine, Host, and Context components are officially termed Containers, and refer to components that process incoming requests and generate an appropriate outgoing response.

Nested Components can be thought of as sub-elements that can be nested inside either Top Level Elements or other Containers to configure how they function. Examples of nested components include the Valve, which represents a reusable unit

of work; the Pipeline, which represents a chain of Valves strung together; and a Realm which helps set up container-managed security for a particular container. Other nested components include the Loader which is used to enforce the specification's guidelines for servlet class loading; the Manager that supports session management for each web application; the Resources component that represents the web application's static resources and a mechanism to access these resources; and the Listener that allows you to insert custom processing at important points in a container's life cycle, such as when a component is being started or stopped. Not all nested components can be nested within every container. A final major component, which falls into its own category, is the Connector. It represents the connection end point that an external client (such as a web browser) can use to connect to the Tomcat container.



Architectural benefits:

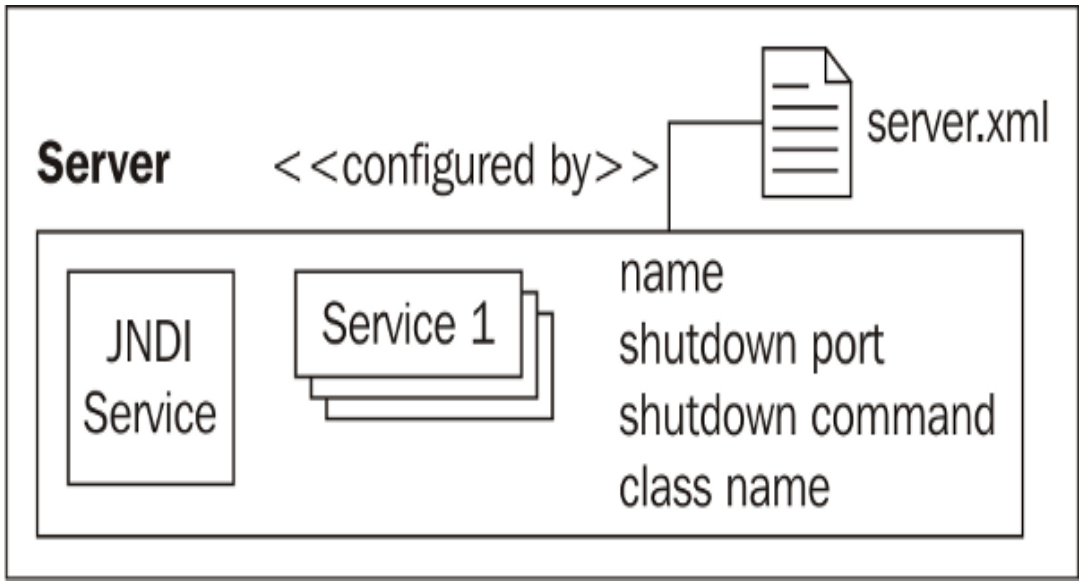
This architecture has a couple of useful features. It not only makes it easy to manage component life cycles (each component manages the life cycle notifications for its children), but also to dynamically assemble a running Tomcat server instance that is based on the information that has been read from configuration files at startup. In particular, the server.xml file is parsed at startup, and its contents are used to instantiate and configure the defined elements, which are then assembled into a running Tomcat instance. The server.xml file is read only once, and edits to it will not be picked up until Tomcat is restarted. This architecture also eases the configuration burden by allowing child containers to inherit the configuration of

their parent containers. For instance, a Realm defines a data store that can be used for authentication and authorization of users who are attempting to access protected resources within a web application. For ease of configuration, a realm that is defined for an engine applies to all its children hosts and contexts. At the same time, a particular child, such as a given context, may override its inherited realm by specifying its own realm to be used in place of its parent's realm.

Top Level Components:

The Server and Service container components exist largely as structural conveniences. A Server represents the running instance of Tomcat and contains one or more Service children, each of which represents a collection of request processing components.

Server: A Server represents the entire Tomcat instance and is a singleton within a Java Virtual Machine, and is responsible for managing the life cycle of its contained services. The following image depicts the key aspects of the Server component. As shown, a Server instance is configured using the server.xml configuration file. The root element of this file is <Server> and represents the Tomcat instance. Its default implementation is provided using org.apache.catalina.core.StandardServer, but you can specify your own custom implementation through the class Name attribute of the <Server> element.

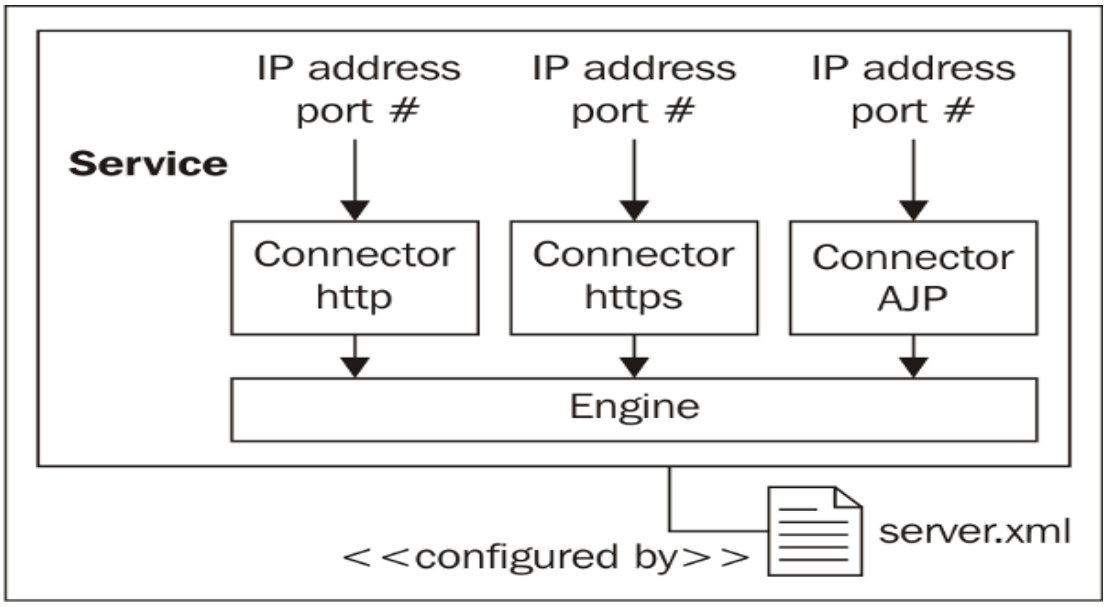


A key aspect of the Server is that it opens a server socket on port 8005 (the default) to listen a shutdown command (by default, this command is the text string SHUTDOWN). When this shutdown command is received, the server gracefully shuts itself down. For security reasons, the connection requesting the shutdown must be initiated from the same machine that is running this instance of Tomcat. A Server also provides an implementation of the Java Naming and Directory Interface (JNDI) service, allowing you to register arbitrary objects (such as data sources) or environment variables, by name. At runtime, individual components (such as servlets) can retrieve this information by looking up the desired object name in the server's JNDI bindings. While a JNDI implementation is not integral to the functioning of a servlet container, it is part of the Java EE specification and is a service that servlets have a right to expect from their application servers or servlet containers. Implementing this service makes for easy portability of web applications across containers. While there is always just one server instance within a JVM, it is entirely possible to have multiple server instances running on a single physical machine, each encased in its own JVM. Doing so insulates web applications that are running on one VM from errors in applications that are running on others, and

simplifies maintenance by allowing a JVM to be restarted independently of the others. This is one of the mechanisms used in a shared hosting environment (the other is virtual hosting, which we will see shortly) where you need isolation from other web applications that are running on the same physical server.

Service:

While the Server represents the Tomcat instance itself, a Service represents the set of request processing components within Tomcat. A Server can contain more than one Service, where each service associates a group of Connector components with a single Engine. Requests from clients are received on a connector, which in turn funnels them through into the engine, which is the key request processing component within Tomcat. The image shows connectors for HTTP, HTTPS, and the Apache JServ Protocol (AJP). There is very little reason to modify this element, and the default Service instance is usually sufficient.



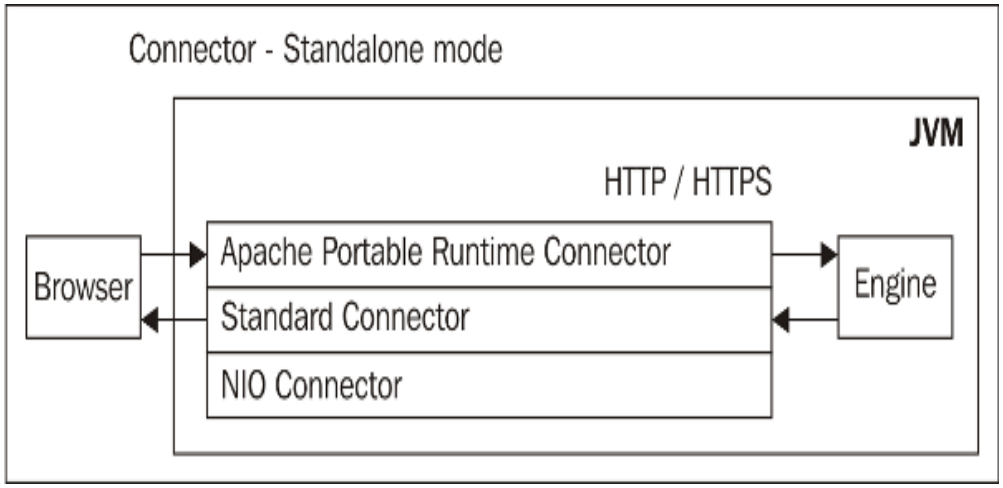
A hint as to when you might need more than one Service instance can be found in the above image. As shown, a service aggregates connectors, each of which monitors

a given IP address and port, and responds in a given protocol. An example use case for having multiple services, therefore, is when you want to partition your services (and their contained engines, hosts, and web applications) by IP address and/or port number.

For instance, you might configure your firewall to expose the connectors for one service to an external audience, while restricting your other service to hosting intranet applications that are visible only to internal users. This would ensure that an external user could never access your Intranet application, as that access would be blocked by the firewall. The Service, therefore, is nothing more than a grouping construct. It does not currently add any other value to the proceedings.

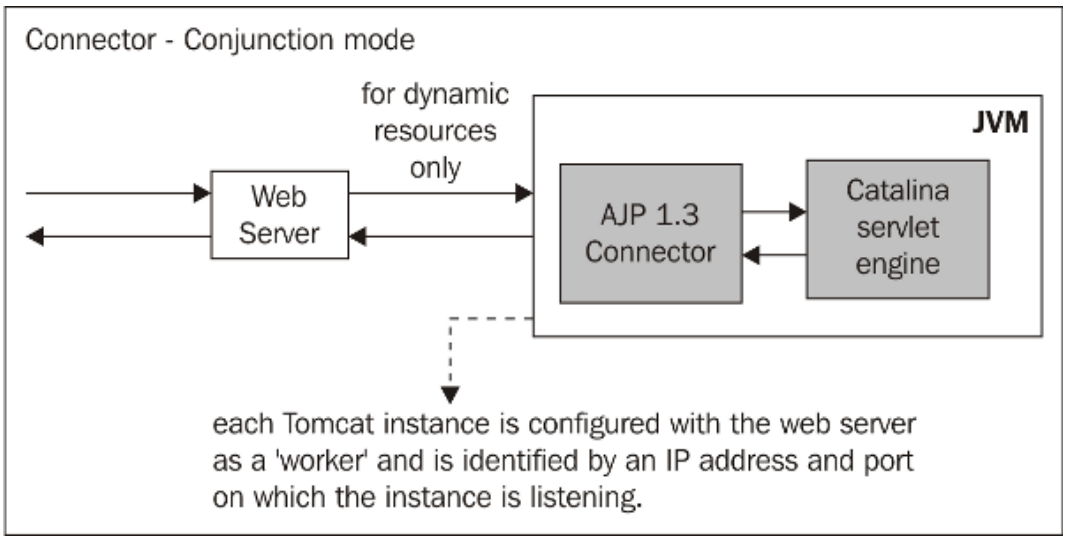
Connectors:

A Connector is a service endpoint on which a client connects to the Tomcat container. It serves to insulate the engine from the various communication protocols that are used by clients, such as HTTP, HTTPS, or the Apache JServ Protocol (AJP). Tomcat can be configured to work in two modes—Standalone or in Conjunction with a separate web server. In standalone mode,



Tomcat is configured with HTTP and HTTPS connectors, which make it act like a full-fledged web server by serving up static content when requested, as well as by delegating to the Catalina engine for dynamic content. Out of the box, Tomcat provides three possible implementations of the HTTP/1.1 and HTTPS connectors for this mode of operation. The most common are the standard connectors, known as Coyote which are implemented using standard Java I/O mechanisms. You may also make use of a couple of newer implementations, one which uses the non-blocking NIO features of Java 1.4, and the other which takes advantage of native code that is optimized for a particular operating system through the Apache Portable Runtime (APR). Note that both the Connector and the Engine run in the same JVM.

In fact, they run within the same Server instance. In conjunction mode, Tomcat plays a supporting role to a web server, such as Apache http or Microsoft's IIS. The client here is the web server, communicating with Tomcat either through an Apache module or an ISAPI DLL. When this module determines that a request must be routed to Tomcat for processing, it will communicate this request to Tomcat using AJP, a binary protocol that is designed to be more efficient than the text based HTTP when communicating between a web server and Tomcat. On the Tomcat side, an AJP connector accepts this communication and translates it into a form that the Catalina engine can process.

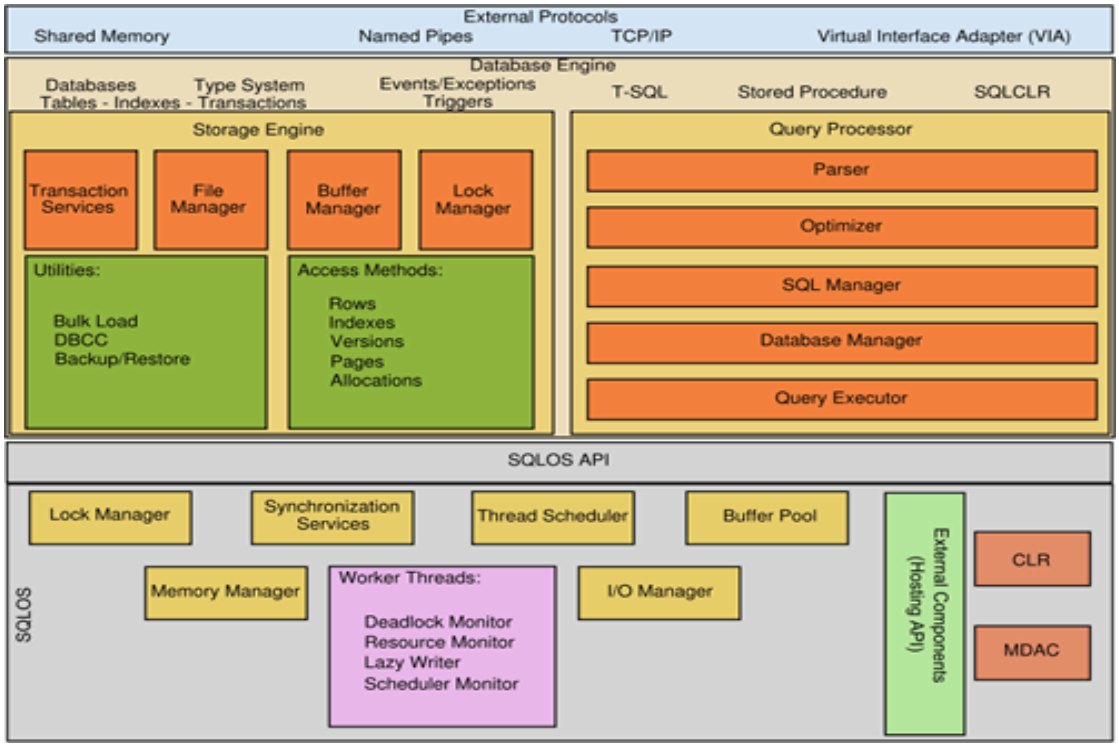


In this mode, Tomcat is running in its own JVM as a separate process from the web server. In either mode, the primary attributes of a Connector are the IP address and port on which it will listen for incoming requests, and the protocol that it supports. Another key attribute is the maximum number of request processing threads that can be created to concurrently handle incoming requests. Once all these threads are busy, any incoming request will be ignored until a thread becomes available. By default, a connector listens on all the IP addresses for the given physical machine (its address attribute defaults to 0.0.0.0). However, a connector can be configured to listen on just one of the IP addresses for a machine. This will constrain it to accept connections from only that specified IP address. Any request that is received by any one of a service's connectors is passed on to the service's single engine. This engine, known as Catalina, is responsible for the processing of the request, and the generation of the response. The engine returns the response to the connector, which then transmits it back to the client using the appropriate communication protocol.

MYSQL Server

Microsoft MYSQL Server is a relational database management system developed by Microsoft. As a database, it is a software product whose primary function is to store and retrieve data as requested by other software applications, be it those on the same computer or those running on another computer across a network (including the Internet). There are at least a dozen different editions of Microsoft MYSQL Server aimed at different audiences and for different workloads (ranging from small applications that store and retrieve data on the same computer, to millions of users and computers that access huge amounts of data from the Internet at the same time). True to its name, Microsoft MYSQL Server's primary query languages are T-MYSQL and ANSI MYSQL.

MYSQL Server Architecture Diagram:



For this purpose, it defined an xml data type that could be used either as a data type in database columns or as literals in queries.

XML columns can be associated with XSD schemas; XML data being stored is verified against the schema. XML is converted to an internal binary data type before being stored in the database. Specialized indexing methods were made available for XML data. XML data is queried using XQuery; MYSQL Server 2005 added some extensions to the T-MYSQL language to allow embedding XQuery queries in T-MYSQL. In addition, it also defines a new extension to XQuery, called XML DML that allows query-based modifications to XML data. MYSQL Server 2005 also allows a database server to be exposed over web services using Tabular Data Stream (TDS) packets encapsulated within SOAP (protocol) requests. When the data is accessed over web services, results are returned as XML.

Common Language Runtime (CLR) integration was introduced with this version, enabling one to write MYSQL code as Managed Code by the CLR. For relational data, T-MYSQL has been augmented with error handling features (try/catch) and support for recursive queries with CTEs (Common Table Expressions). MYSQL Server 2005 has also been enhanced with new indexing algorithms, syntax and better error recovery systems. Data pages are check summed for better error resiliency, and optimistic concurrency support has been added for better performance. Permissions and access control have been made more granular and the query processor handles concurrent execution of queries in a more efficient way. Partitions on tables and indexes are supported natively, so scaling out a database onto a cluster is easier.

MYSQL CLR was introduced with MYSQL Server 2005 to let it integrate with the .NET Framework. MYSQL Server 2005 introduced "MARS" (Multiple Active Results Sets), a method of allowing usage of database connections for multiple purposes. MYSQL Server 2005 introduced DMVs (Dynamic Management Views), which are specialized views and functions that return server state

information that can be used to monitor the health of a server instance, diagnose problems, and tune performance. Service Pack 1 (SP1) of MYSQL Server 2005 introduced Database Mirroring, a high availability option that provides redundancy and failover capabilities at the database level. Failover can be performed manually or can be configured for automatic failover.

CHAPTER 5

5.1 SYSTEM TESTING

FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

5.1.1 SYSTEM TESTING AND MAINTENANCE

Testing is vital to the success of the system. System testing makes a logical assumption that if all parts of the system are correct, the goal will be successfully achieved. In the testing process we test the actual system in an organization and gather errors from the new system operates in full efficiency as stated. System testing is the stage of implementation, which is aimed to ensuring that the system works accurately and efficiently.

In the testing process we test the actual system in an organization and gather errors from the new system and take initiatives to correct the same. All the front-end and back-end connectivity are tested to be sure that the new system operates in full efficiency as stated. System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently.

The main objective of testing is to uncover errors from the system. For the uncovering process we have to give proper input data to the system. So we should have more conscious to give input data. It is important to give correct inputs to efficient testing.

Testing is done for each module. After testing all the modules, the modules are integrated and testing of the final system is done with the test data, specially designed to show that the system will operate successfully in all its aspects conditions. Thus the system testing is a confirmation that all is correct and an opportunity to show the user that the system works.

This will create two problems, Time delay between the cause and appearance of the problem. The effect of the system errors on files and records within the system.

The purpose of the system testing is to consider all the likely variations to which it will be suggested and push the system to its limits

The testing process focuses on logical intervals of the software ensuring that all the statements have been tested and on the function intervals (i.e.,) conducting tests to uncover errors and ensure that defined inputs will produce actual results that agree with the required results. Testing has to be done using the two common steps Unit testing and Integration testing. In the project system testing is made as follows:

The procedure level testing is made first. By giving improper inputs, the errors occurred are noted and eliminated. This is the final step in system life cycle. Here we implement the tested error-free system into real-life environment and make necessary changes, which runs in an online fashion. Here system maintenance is done every months or year based on company policies, and is checked for errors like runtime errors, long run errors and other maintenances like table verification and reports.

5.2 UNIT TESTING

Unit testing verification efforts on the smallest unit of software design, module. This is known as “Module Testing”. The modules are tested separately. This testing is carried out during programming stage itself. In these testing steps, each module is found to be working satisfactorily as regard to the expected output from the module.

5.3 INTEGRATION TESTING

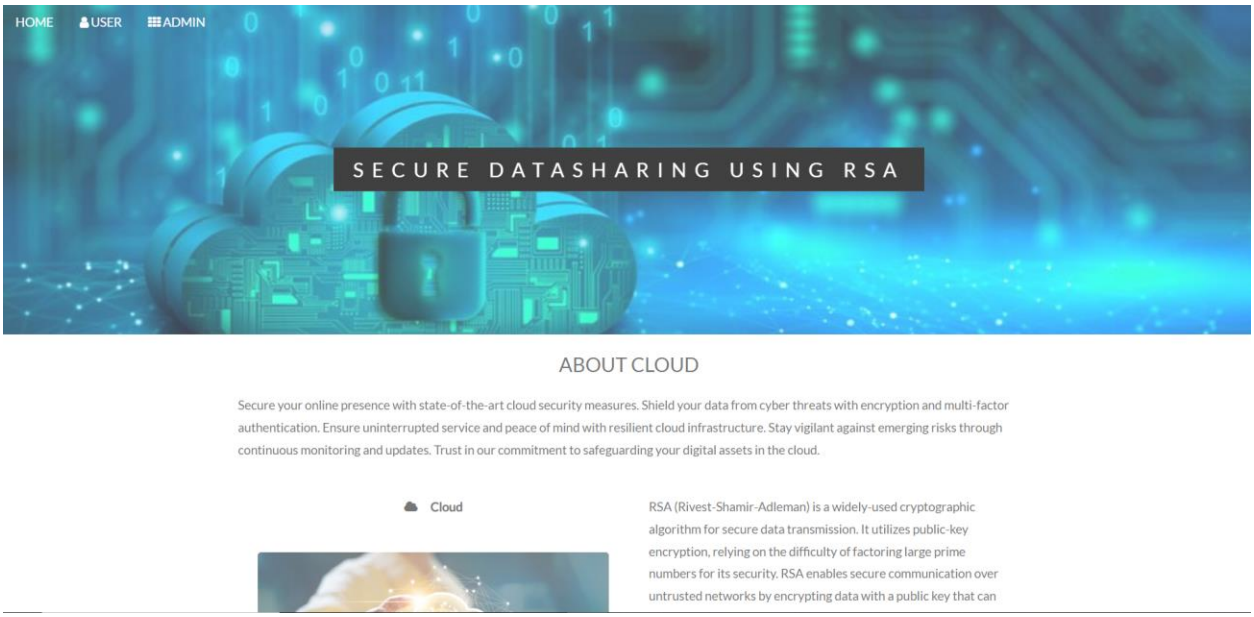
Integration testing is a systematic technique for constructing tests to uncover error associated within the interface. In the project, all the modules are combined and then the entire programmer is tested as a whole. In the integration-testing step, all the error uncovered is corrected for the next testing steps.

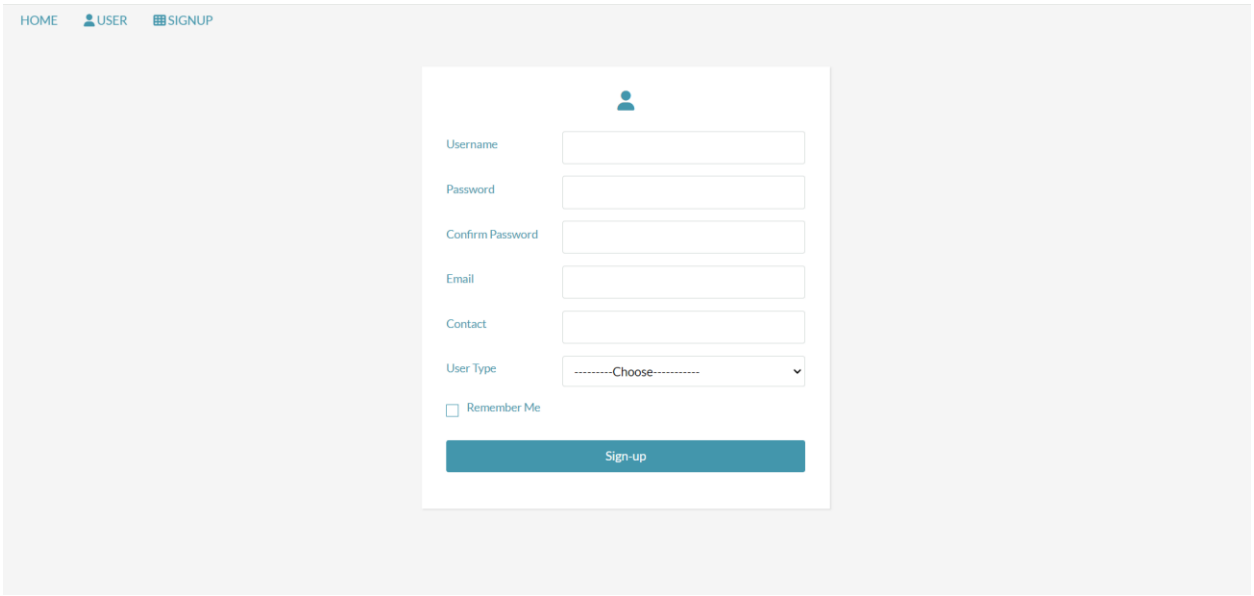
CHAPTER-6

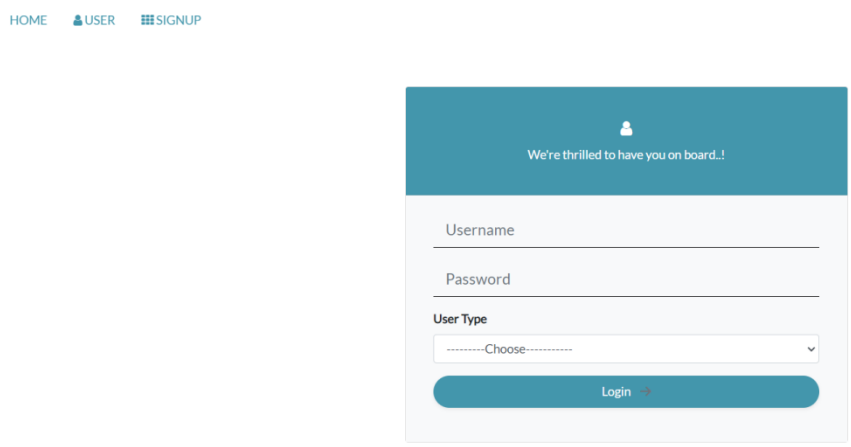
6.1 CONCLUSION:

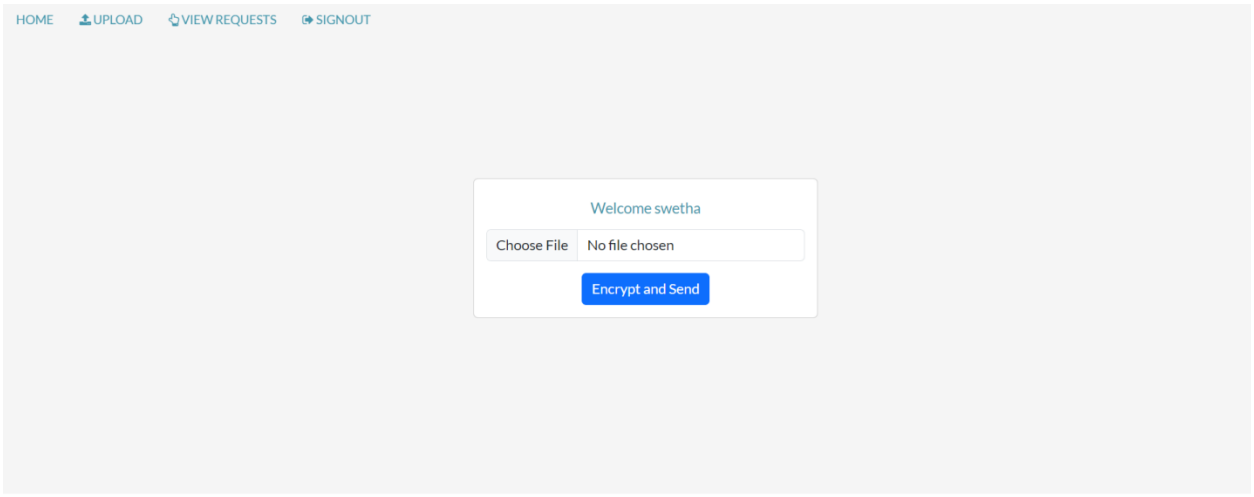
In this paper we have proposed the first mCL-PKE scheme without pairing operations and provided its formal security. Our mCL-PKE solves the key escrow problem and revocation problem. Using the mCLPKE scheme as a key building block, we proposed an improved approach to securely share sensitive data in public clouds. Our approach supports immediate revocation and assures the confidentiality of the data stored in an untrusted public cloud while enforcing the access control policies of the data owner. Our experimental results show the efficiency of basic mCLPKE scheme and improved approach for the public cloud. Further, for multiple users satisfying the same access control policies, our improved approach performs only a single encryption of each data item and reduces the overall overhead at the data owner.

6.2 SCREENSHOTS:






HOME
USER
SIGNUP
We're thrilled to have you on board..!
Username
Password
User Type
---------Choose----------
Login


HOME
UPLOAD
VIEW REQUESTS
SIGNOUT
Welcome swetha
Choose File
No file chosen
Encrypt and Send

HOME UPLOAD VIEW REQUESTS SIGNOUT

REQUEST DETAILS

Consumer Id	Consumer Name	File Name	Requested Date	Accept/Reject
11	swe	doctor.txt	2024-04-09 12:37:24	Change

SEND REQUEST VIEW REQUEST DETAILS SIGNOUT

FILE DETAILS

File ID	File Name	File In	Public Key	Owner ID	Send Request
12	3.txt	VM1 or Cloud1	21015	7	Click Here
13	doctor (1).txt	VM1 or Cloud1	28191	9	Click Here
14	doctor.txt	VM1 or Cloud1	22215	10	Click Here

SEND REQUEST VIEW REQUEST DETAILS SIGNOUT

Requested File Details

Requested Date	File Name	Request Status	Download
2024-04-09 12:37:24	doctor.txt	Allow	Download

6.3 CODING:

Signup.jsp:

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Efficient Encryption</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<link href="style.css" rel="stylesheet" type="text/css" />
<!-- CuFon: Enables smooth pretty custom font rendering. 100% SEO friendly. To disable, remove this section -->
<script type="text/javascript" src="js/cufon-yui.js"></script>
<script type="text/javascript" src="js/arial.js"></script>
<script type="text/javascript" src="js/cuf_run.js"></script>
<!-- CuFon ends -->
</head>
<body>
<div class="main">

  <div class="header">
```

```
    <div class="header_resize">
      <div class="logo"><h2><a href="index.html">Efficient Encryption</a></h2></div>
      <div class="menu_nav">
        <div class="clr"></div>
        <ul>
          <li class="active"><a href="index.html">Home</a></li>
          <li><a href="userlogin.jsp">User</a></li>
          <li><a href="signup.jsp">Sign Up</a></li>
        </ul>
      </div>
      <div class="clr"></div>
    </div>
  </div>

  <div>
    <div class="hbg_resize">
      <form action="signupcheck.jsp" method="post">
        <br></br>
```

```
<center><h2>Sign Up Details</h2></center>

<table width="300" align="center" >

<tr><td style="color: black">User Name</td><td><input type="text" name="username" required /></td><td></td></tr>

<tr><td style="color: black">Password</td><td><input type="password" name="password"  MaxLength="7" required /> </td><td></td></tr>

<tr><td style="color: black">Confirm Password</td><td><input type="password" name="repwd" MaxLength="7" required/></td><td></td></tr>

<tr><td style="color: black">Email</td><td><input type="text" name="email"  required/></td><td></td></tr>

<tr><td style="color: black">Contact no</td><td><input type="text" name="phone"  MaxLength="10" required/></td><td></td></tr>

<tr><td style="color: black;">User Type</td><td><select name="usertype" style="width:174px">

<option>---------Choose----------</option>

<option>Data Owner</option>

<option> Data User</option>

</select></td></tr>

<tr><td style="color: black">Cluster</td><td><select name="cluster" style="width:174px">

<option>---------Choose----------</option>
```

```
                <option>Cluster1</option>

                <option>Cluster2</option>

                <option>Cluster3</option>

                <option>Cluster4</option>


                 </select></td></tr>


        <tr><td></td><td>

                <br>

                <input type="submit" value="Submit" style="width: 110px;
  color: white;
  background-color: #3a90c9;
  font-weight: bold;
  height: 41px;
  border-radius: 100px;"  /></td><td></td></tr>
   </table>
 </form>
  </div>
</div>
```

```
  </div>
</body>
</html>
```

Ownerpage.jsp:

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Efficient Encryption</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<link href="style.css" rel="stylesheet" type="text/css" />
<!-- CuFon: Enables smooth pretty custom font rendering. 100% SEO friendly. To disable, remove this section -->
<script type="text/javascript" src="js/cufon-yui.js"></script>
<script type="text/javascript" src="js/arial.js"></script>
```

```
<script type="text/javascript" src="js/cuf_run.js"></script>
<!-- CuFon ends -->


</head>
<body>
<div class="main">
  <div class="header">
    <div class="header_resize">
      <div class="logo"><h2><a href="index.html">Efficient Encryption</a></h2></div>
      <div class="menu_nav">
        <div class="clr"></div>
        <ul>
          <li class="active"><a href="ownerpage.jsp">Home</a></li>
          <li><a href="ownerpage.jsp">Upload</a></li>
          <li><a href="viewrequest.jsp">View Requests</a></li>
          <li><a href="index.html">Sign Out</a></li>
        </ul>
      </div>
```

```
    <div class="clr"></div>
   </div>
  </div>
  <div class="hbg">
   <div class="hbg_resize">
    <form align="center" ENCTYPE="multipart/form-data" ACTION="encrypt.jsp" METHOD=POST>
        <br><br></br> <table height="200"  >
          <tr>

            <td align="right">
              <img src="images/ff.jpg" width="250" height="250" alt="admin_login"/>
            </td>
            <td>
              <%
              String name=(String)session.getAttribute("name1");
              session.setAttribute("name1",name);
              System.out.println("owner is"+name);
              int id=(Integer)session.getAttribute("userid");
```

```
                session.setAttribute("userid",id);
               %>
               <table>
<tr align="center" ><h2 style="text-align: center;color: #323a3f;font-size: medium;font-weight: bold">Welcome <span><%=name%></span></h2></tr>
<br></br>
<tr>
        <td></td>
        <td style="color: #cc0000">
            Select VM
        </td>
        <td>
           <select name="vmachine">
               <option value="select">.....Select.....</option>
               <option value="VM1 or Cloud1">VM1 or Cloud1</option>
               <option value="VM2 or Cloud2">VM2 or Cloud2</option>
               <option value="VM3 or Cloud3">VM3 or Cloud3</option>
               <option value="VM4 or Cloud4">VM4 or Cloud4</option>
           </select>
```

```
        </td>
    </tr>
    <tr>
        <td>
        </td>
        <td style="color: black;font-family: sans-serif;font-weight: bold;font-size: small" >
            Upload File:
        </td>
        <td>
            <input type="file" name="file1" value="" style="color: #323a3f"/>
        </td>

    </tr>
 <tr></tr><tr></tr>
    <tr>
        <td></td>
        <td>
            <input type="submit" value="Encrypt" style="width:80px; color: white;background-color: blue;font-weight: bold" />
```

```
        </td>
      </tr>
    </table>
          </td>
        </tr>

    </table>
  </form>
  </div>
</div>

<div class="content">
  <div class="content_resize">
    <div class="mainbar">
  </div>
    <div class="sidebar">
    </div>
    <div class="clr"></div>
  </div>
```

```
  </div><br><br></br><br></br>
  <div class="footer">
    <div class="footer_resize">
      <p class="lf">&copy; Copyright MyWebSite. Designed by Blue <a href="#">Website Templates</a></p>
      <ul class="fmenu">
        <li class="active"><a href="#">Home</a></li>
        <li><a href="#">Support</a></li>
        <li><a href="#">Blog</a></li>
        <li><a href="#">About Us</a></li>
        <li><a href="#">Contacts</a></li>
      </ul>
      <div class="clr"></div>
    </div>
  </div>
</div>
</body>
</html>
```

download_files.jsp:

```
<%@include file="getcon.jsp" %>

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Efficient Encryption</title>

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

<link href="style.css" rel="stylesheet" type="text/css" />

<!-- CuFon: Enables smooth pretty custom font rendering. 100% SEO friendly. To disable, remove this section -->

<script type="text/javascript" src="js/cufon-yui.js"></script>

<script type="text/javascript" src="js/arial.js"></script>

<script type="text/javascript" src="js/cuf_run.js"></script>

<!-- CuFon ends -->

<script>

var $table = $('table.scroll'),

    $bodyCells = $table.find('tbody tr:first').children(),

    colWidth;


// Adjust the width of thead cells when window resizes
```

```
$(window).resize(function() {
    // Get the tbody columns width array
    colWidth = $bodyCells.map(function() {
        return $(this).width();
    }).get();

    // Set the width of thead columns
    $table.find('thead tr').children().each(function(i, v) {
        $(v).width(colWidth[i]);
    });
}).resize(); // Trigger resize handler
</script>
</head>
    <body >
<div class="main">
  <div class="header">
    <div class="header_resize">
      <div class="logo"><h2><a href="index.html">Efficient Encryption</a></h2></div>
```

```
      <div class="menu_nav">
        <div class="clr"></div>
        <ul>
          <li class="active"><a href="userpage.jsp">Home</a></li>
          <li><a href="download_files.jsp">Download from cloud</a></li>
          <li><a href="sendrequest.jsp">Send Request</a></li>
          <li><a href="requeststatus.jsp">View Request Details</a></li>
          <li><a href="index.html">Sign Out</a></li>
        </ul>
      </div>
      <div class="clr"></div>
    </div>
  </div>
<div class="hbg" >
    <div class="hbg_resize">
      <FORM NAME="form1" METHOD="POST">
<%
                        try {
                             Statement st = con.createStatement();
```

```
ResultSet resultset1 =

    st.executeQuery("select * from filedetails");


%>

<center>  <TABLE class="scroll" style="border-color: #d7dbe1; " border="1" cellspacing="5" cellpadding="5" width="700" rules="all">

<h2 style="font-size: x-large;color: #cc0066; font-family: sans-serif " align="center">File Details</h2>

<tr style="font-size:large;color: #0000cc;font-family: serif">

<TH  style=" text-align: center"  >File ID</TH>

<TH  style=" text-align: left" >File Name</TH>

<TH  style=" text-align: left" >File In</TH>

<TH  style=" text-align: left" >Public Key</TH>

<TH  style=" text-align: left" >Owner ID</TH>

<TH  style=" text-align: left" >Download</TH>

</tr>

<% while (resultset1.next()) {%>

<tr style="font-size:large;color: #0000cc;font-family: serif">

<TD style=" text-align: center" ><%= resultset1.getString("fileid")%> </TD>
```

```
                    <TD style=" text-align: left" ><%= resultset1.getString("filename")%></TD>
                    <TD style=" text-align: left" ><%= resultset1.getString("virtual_machine")%></TD>
                    <TD style=" text-align: left" ><%= resultset1.getString("public_key")%></TD>
                    <TD style=" text-align: left" ><%= resultset1.getString("ownerid")%></TD>
                    <TD style=" text-align: left" ><a href="nowdownload.jsp?id=<%= resultset1.getString("fileid")%>">Click Here</a></TD>
                    </tr>

                    <% }%>
                    </TABLE> </center>
                <%
} catch (Exception e) {
                    }
%>
</FORM>
   </div>
```

```
  </div>
  <div class="content">
    <div class="content_resize">
      <div class="mainbar">


</div>
      <div class="sidebar">
  </div>
      <div class="clr"></div>
    </div>
  </div>
</div>
</body>
</html>
```

6.4 REFERENCES:

[1] M. Abdalla, M. Bellare, D. Catalano, E. Kiltz, T. Kohno, T. Lange, J. Malone-Lee, G. Neven, P. Paillier, and H. Shi. Searchable encryption revisited: Consistency properties, relation to anonymous ibe, and extensions. J. Cryptol., 21(3):350– 391, Mar. 2008.

[2] S. Al-Riyami and K. Paterson. Certificateless public key cryptography. In C.-S. Laih, editor, Advances in Cryptology - ASIACRYPT 2003, volume 2894 of Lecture Notes in Computer Science, pages 452–473. Springer Berlin / Heidelberg, 2003.

[3] M. Bellare, D. Pointcheval, and P. Rogaway. Relations among notions of security for public-key encryption schemes. pages 26–45. Springer-Verlag, 1998.

[4] E. Bertino and E. Ferrari. Secure and selective dissemination of XML documents. ACM Transactions on Information and System Security, 5(3):290–331, 2002.

[5] J. Bethencourt, A. Sahai, and B. Waters. Ciphertext-policy attribute-based encryption. In SP ’07: Proceedings of the 2007 IEEE Symposium on Security and Privacy, pages 321–334, Washington, DC, USA, 2007. IEEE Computer Society.

[6] D. Boneh, X. Ding, and G. Tsudik. Fine-grained control of security capabilities. ACM Transactions on Internet Technology, 4(1):60–82, Feb. 2004.